App Dev 2 Project Report

1. Student Details

Name: Dhruvil Patel
Roll Number: 25f2001780
Email: 25f2001780@ds.study.iitm.ac.in
About Me: I am a student in the IIT Madras BS Degree program with an interest in full-stack web development and database management. I enjoy building secure, role-based applications that solve real-world administrative and logistical challenges. Also perusing offline B.Tech in Computer Science Engineering at Nirma University Ahmedabad.

---

2. Project Details

- **Project Title:** Campus Placement Portal Application V2
- **Problem Statement:** Institutes currently rely on manual processes, spreadsheets, and emails to manage campus recruitment, making it difficult to track student applications, manage company approvals, and prevent duplicate entries.
- **Approach:** I built a role-based, centralized Single Page Application (SPA) using Vue.js and Flask. The system strictly separates functionalities for three user types: Admins (Institute), Companies, and Students. It features a secure JWT backend validation system, dynamic interactive dashboards, real-time application tracking, and an asynchronous worker queue (Celery) for heavy tasks to streamline the entire recruitment lifecycle.

---

3. AI/LLM Declaration

- The extent of AI/LLM usage is approximately 10–15%, strictly limited to minor CSS design assistance and schedule syntax.
- All final business logic, database integration, API endpoints, security validations, and extensive testing were implemented and verified manually from scratch.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework and REST API routing
Vue.js 3	Frontend Single Page Application (SPA) framework
SQLAlchemy	Object Relational Mapper (ORM) for database interactions
SQLite	Lightweight local database for storing application data
Celery & Redis	Asynchronous background task processing and scheduling
Flask-JWT-Extended	Secure token-based user authentication and authorization
Bootstrap 5	Frontend styling, responsive design, and CSS framework

Axios & Chart.js	API communication and dynamic admin analytics visualizations

---

5. Database Schema / ER Diagram

Tables:

- **User**: Core authentication records (id, email, phone, password, role, active, created_at).
- **StudentProfile**: Stores student academic profiles
  (id, user_id, name, branch, cgpa, year, resume_file_path, profile_photo_path).
- **CompanyProfile**: Stores recruiter profiles
  (id, user_id, name, description, hr_name, hr_contact, website, logo_path, approved).
- **PlacementDrive**: Logs job postings created by companies
  (id, company_id, job_title, description, package_lpa, location, branch_eligibility, cgpa_cutoff, appl ication_deadline, status, created_at).
- **Application**: Tracks student applications to drives
  (id, student_id, drive_id, application_date, status).

Relationships:

- One-to-One → **User → StudentProfile / CompanyProfile** (Each user has one specific role profile)
- One-to-Many → **CompanyProfile → PlacementDrive** (A company can host multiple drives)
- One-to-Many → **StudentProfile → Application** (A student can apply to multiple drives)
- One-to-Many → **PlacementDrive → Application** (A drive can have multiple student applications)

---

6. Key Application Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Authenticates user credentials and returns a JWT token
/api/auth/register	POST	Validates and registers a new student or company account
/api/company/drives	POST	Allows approved company to create a new placement drive
/api/student/drives/<id>/apply	POST	Creates an application record for a student
/api/admin/users/<id>/toggle	POST	Admin action to activate/deactivate an account
/api/student/export-history	POST	Triggers a Celery background task to export CSV history

---

7. Architecture and Features

Architecture Overview:

- backend/app.py – Main application entry point, Celery initialization, and API blueprint registration.

- backend/models/models.py – Database schemas defined using SQLAlchemy.
- backend/routes/ – Modularized Flask blueprints separated by user roles (admin.py, company.py, student.py, auth.py).
- backend/tasks/celery_tasks.py – Asynchronous workers for heavy data processing and automated emails.
- frontend/src/ – Vue 3 SPA architecture containing reusable components and dynamic views (AdminDashboard.vue, etc.).

Implemented Core Features:

- Strict Role-Based Access Control (Admin, Company, Student) secured by JWT.
- Dynamic Placement Drive creation and management.
- Real-time Student Application tracking and status updates.
- Admin dashboard with Chart.js analytics, caching (Redis), and live search filtering.

Additional / Advanced Features:

- **Asynchronous Processing (Celery):** Offloads heavy tasks like CSV generation and report building to background workers to maintain a responsive UI.
- **Intelligent Eligibility Engine:** Automatically hides drives from students who do not meet the minimum CGPA or branch requirements server-side.
- **Automated Task Scheduling:** Uses Celery Beat to automatically check deadlines and send email reminders to students.
- **Caching Mechanism:** Caches frequently accessed statistics and drive lists using Redis to optimize database loads and reduce latency.

8. Video Presentation

- **Drive Link:** https://drive.google.com/file/d/1xDsWdgWgVS3zu-QGkQKQYBzlmyo7ttJh/view?usp=sharing